

FAPO: Fanout-Aware Post-Mapping Optimization for LUT-Based FPGAs

Xiaoyu Hao, Keren Zhu*

State Key Lab of Integrated Chips and Systems, Fudan University, China

Abstract—Reducing logic depth during FPGA technology mapping does not necessarily eliminate fanout bottlenecks in the resulting LUT networks. Replicating heavily loaded LUTs can alleviate these bottlenecks, but the added copies consume area and may increase upstream loading. We propose FAPO, a mapper-independent post-mapping optimizer that jointly refines sink assignments and LUT cuts under a final-area constraint. Replication, cut switching, and recovery are evaluated by the resulting live LUT count. We derive a necessary all-critical-path condition for delay reduction and obtain minimum-copy proposals by minimum cut within a restricted same-cut family. These proposals complement greedy refinement, with mappings selected by full-network area and timing. Experiments on EPFL benchmarks with three initial mappers achieve geometric-mean reductions of up to 14.48% in fanout-aware area–timing product (ATP) with a 5% final-area allowance. VPR evaluation shows a 7.04% reduction in routed critical-path delay relative to ABC *if*.

Index Terms—FPGA technology mapping, lookup tables, post-mapping optimization, logic replication, fanout-aware timing, area–timing product

I. INTRODUCTION

Lookup-table (LUT) technology mapping determines the area and timing of field-programmable gate arrays (FPGAs). Classical methods optimize depth and area through network-flow and cut-based techniques [1]–[6]. However, structural depth alone is an incomplete timing proxy [7]. Networks with the same number of logic levels can place different loads on their LUTs. A heavily shared LUT may drive many sinks, creating a fanout bottleneck that depth reduction alone does not resolve. Placement-driven mapping uses physical delay estimates [8], while WireMap reduces pin-to-pin connections to improve routability [9].

Recent methods explore adaptive gate decomposition, Ashenhurst–Curtis decomposition, equality saturation, and heterogeneous representations [10]–[13]. Resubstitution and mapping fusion further improve local choices [14], [15]. These methods broaden the mapping search space, but their resulting covers can still retain concentrated sink loads. Once a legal LUT network is available, its sink assignments and selected cuts provide a common interface for further optimization. Such refinement can complement different mapping flows without depending on their internal representations or search histories.

Prior work reduces fanout through gate duplication [16] and couples replication with FPGA placement [17]. The challenge

is to coordinate sink assignments and LUT cuts within a final LUT budget. Replication partitions sinks but can shift the bottleneck upstream by increasing fanin loads. Cut switching changes these dependencies; cleanup and recovery remove redundant instances. FAPO evaluates these coupled changes by final live area and timing, allowing recovery to offset replication costs.

Strict delay reduction requires affecting every current critical path; changing only one of several tied paths can leave the maximum unchanged. FAPO combines minimum-cut proposals on the complete critical DAG with greedy compound refinement, remeasuring the resulting network to account for changed loads. Our main contributions are as follows:

- We formulate joint sink-assignment and LUT-cut optimization over a mapper-independent state, charging compound replication, cut switching, and recovery by final live LUT count.
- We establish a necessary all-critical-path condition and show that minimum cut finds the smallest same-cut copy set intersecting every current critical path.
- We integrate one bounded minimum-cut proposal per sweep with greedy compound refinement, using remeasured area and timing to admit candidates and select minimum ATP.
- Across three mappers, FAPO lowers geometric-mean ATP by up to 14.48% with a 5% final-area allowance. VPR evaluation shows a 7.04% reduction in routed critical-path delay relative to ABC *if*.

II. MAPPED-NETWORK OPTIMIZATION

A. State and final-area constraint

Let $M_0 = \mathcal{B}(G)$ be a legal K -LUT mapping of a combinational source network G . The optimizer imports M_0 , while G specifies the reference function for equivalence checking [18]. Each live instance is represented by (r, C, S) : its logic root, selected K -feasible cut, and served LUT-input or primary-output sinks. Copies of a root serve disjoint sink sets and may use different cuts. Unreachable instances are removed.

Area $A(M)$ counts live LUT instances after cleanup and recovery. Given $A_0 = A(M_0)$ and allowance β , the output must satisfy

$$A(M) \leq A_{\max} = A_0 + \lfloor \beta A_0 \rfloor. \quad (1)$$

*Corresponding author: Keren Zhu.

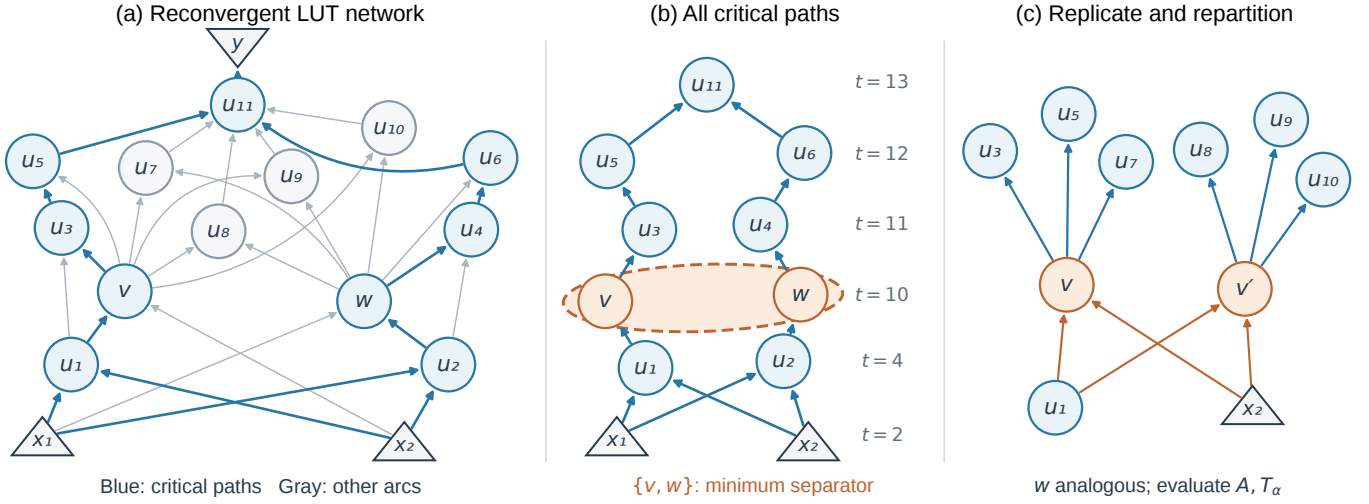


Fig. 1. Illustrative refinement ($K = 6$, $\alpha = 1$). Triangles denote inputs x_i and output y ; circles denote LUT instances. (a) Reconvergent LUT network; critical arcs are blue. (b) Arrivals computed on (a) identify four longest paths, all intersected by $\{v, w\}$; u_i label the other LUTs. (c) Local sink repartition between v and its copy v' ; other fanins are omitted. Apply the same repartition to w and evaluate the resulting area and timing.

Algorithm 1: FAPO post-mapping refinement

Require: Legal seed M_0 , α , β
Ensure: Area-feasible archive $\mathcal{P}$ and selected mapping M^*
1: Set $A_{\max}$ by (1); initialize $\mathcal{F}$ and $\mathcal{A}$ from M_0
2: **repeat**
3: $\mathcal{Q} \leftarrow \text{Evaluate}(\text{Expand}(\mathcal{F}), A_{\max})$
4: $\mathcal{R} \leftarrow \emptyset$
5: **if** the cut branch is enabled **then**
6: $\hat{M} \leftarrow \text{ATPBest}(\text{ParetoSample}(\mathcal{Q}))$
7: $M_c \leftarrow \text{CutBundle}(\hat{M}, A_{\max})$
8: $\mathcal{R} \leftarrow \text{AdmitCut}(M_c, \hat{M}, A_{\max})$
9: **end if**
10: $\mathcal{A} \leftarrow \text{UpdateArchive}(\mathcal{A} \cup \mathcal{Q} \cup \mathcal{R})$
11: $\mathcal{F} \leftarrow \text{UpdateFrontier}(\mathcal{F}, \mathcal{Q})$
12: **until** convergence or the search limit is reached
13: $\mathcal{P} \leftarrow \text{Finalize}(\mathcal{A})$
14: **return** $M^* = \arg \min_{M \in \{M_0\} \cup \mathcal{P}} A(M)T_\alpha(M)$, $\mathcal{P}$

This final-area constraint lets cleanup and recovery offset replication costs, supporting coordinated transformations within the available LUT budget.

FAPO builds its refinement graph directly from the imported LUT cover, without the parent mapper’s internal subject graph or cut database.

B. Fanout-aware timing and objective

The fanout $\text{fo}_M(u)$ counts all live LUT-input arcs and primary-output sinks driven by u . For penalty $g(f) = \max(f - 1, 0)$ and weight $\alpha \geq 0$, arrivals at primary inputs p and LUTs I are

$$t(p) = \alpha g(\text{fo}_M(p)), \quad (2)$$

$$t(I) = 1 + \alpha g(\text{fo}_M(I)) + \max_{J \in \text{fanin}(I)} t(J). \quad (3)$$

The maximum output-driver arrival is $T_\alpha(M)$. This mapping-level timing model captures both logic depth and fanout loading. The search targets

$$\min_{M \equiv G, A(M) \leq A_{\max}} A(M)T_\alpha(M), \quad (4)$$

the fanout-aware area–timing product (ATP), while retaining nondominated (A, T_α) pairs for exploration. The original seed remains a protected candidate. Feasible mappings preserve acyclicity, K -feasibility, and functional equivalence within the area cap.

III. FAPO ALGORITHM

A. Coupled refinement under a final-area bound

FAPO jointly refines sink assignments and LUT cuts in the mapper-independent state (r, C, S) . Replication partitions S between realizations of the same root, reducing their output loads while adding load to shared fanins. Cut switching changes the upstream dependencies of a realization and can make other instances unreachable. Cleanup removes these instances, while copy recovery merges compatible same-root realizations by redirecting their sinks. These interactions make the timing benefit and area cost of each operation depend on the accompanying transformations.

FAPO therefore evaluates each compound transformation after cleanup and recovery, charging the resulting live LUT count against (1). Recovered area can offset replication, allowing useful changes even without net area growth. This final-area accounting couples load redistribution with the choice of LUT realizations: the search can spend area where replication improves timing and recover it through compatible cut changes and copy merges. Full-network evaluation using (2)–(3) captures the resulting upstream loads and critical-path changes.

B. Coordinated replication across critical paths

For fixed M , the timing recurrence defines a node-weighted longest-path problem. Input weights are $\alpha g(\text{fo}_M)$; LUT weights additionally include one logic-delay unit. Writing these weights as $w_M(u)$ gives

$$T_\alpha(M) = \max_P \sum_{u \in P} w_M(u), \quad (5)$$

TABLE I
EPFL MAPPING RESULTS AT $K = 6$, $\alpha = 1$, AND $\beta = 5\%$. PARENT AND FAPO COLUMNS REPORT A/T_1 BEFORE AND AFTER OPTIMIZATION. ATP RATIOS ARE FAPO/PARENT; BOLD VALUES INDICATE IMPROVEMENTS.

Circuit	ABC if			ACD-remap			AGDMap-D		
	Parent	FAPO	ATP ratio	Parent	FAPO	ATP ratio	Parent	FAPO	ATP ratio
adder	254/163	266/135	0.8673	327/68	338/43	0.6536	254/179	266/150	0.8776
bar	512/138	512/138	1.0000	512/138	512/138	1.0000	561/274	564/236	0.8659
div	22113/6483	22132/6052	0.9343	7044/3819	7073/3151	0.8285	19156/9210	19174/8746	0.9505
log2	8092/1091	8118/905	0.8322	8661/1071	8690/921	0.8628	7472/1210	7496/1035	0.8581
max	769/304	774/222	0.7350	786/381	793/157	0.4157	798/367	809/248	0.6851
multiplier	5927/424	5954/403	0.9548	6386/400	6406/373	0.9354	5376/598	5367/446	0.7446
sin	1464/549	1488/426	0.7887	1812/521	1827/387	0.7490	1395/710	1418/534	0.7645
sqrt	5711/11796	5740/9577	0.8160	5963/9526	5992/7398	0.7804	6424/11482	6450/8903	0.7785
square	3998/299	4030/273	0.9204	4135/268	4150/246	0.9212	3592/359	3630/308	0.8670
arbiter	2722/68	2722/68	1.0000	2227/45	2233/43	0.9581	2715/68	2715/68	1.0000
cavlc	116/71	116/71	1.0000	117/68	117/67	0.9853	113/71	113/71	1.0000
ctrl	29/28	29/27	0.9643	29/27	29/27	1.0000	29/29	29/29	1.0000
dec	287/47	287/47	1.0000	284/92	284/92	1.0000	272/248	272/248	1.0000
i2c	353/116	356/93	0.8085	296/120	299/95	0.7997	348/116	351/107	0.9304
int2float	51/31	51/31	1.0000	45/28	46/27	0.9857	45/27	45/27	1.0000
mem_ctrl	12084/1192	12102/1074	0.9023	10236/907	10255/778	0.8594	12104/1730	12124/1370	0.7932
priority	210/66	211/65	0.9895	177/19	177/18	0.9474	227/71	220/66	0.9009
router	89/31	83/17	0.5114	76/16	78/14	0.8980	74/20	77/17	0.8845
voter	2695/72	2709/63	0.8795	1728/45	1738/43	0.9611	2061/66	2066/60	0.9113
Geo. mean ATP ratio	0.8798			0.8552			0.8796		

where P ranges over input-to-output paths. Forward arrivals and reverse tails identify the complete critical DAG $D^*(M)$, including all tied maximum-weight paths.

Coverage condition. Strict delay reduction requires affecting every current critical path: any path retaining its incidences and node weights preserves its previous delay. This motivates selecting coordinated copy sets across $D^*(M)$, since improving only one of several tied paths can leave T_α unchanged.

A same-cut action partitions an eligible LUT's sinks into two nonempty sets served by the original and one copy with identical root, cut, and fanins. Each action introduces one LUT before recovery.

Minimum-copy coverage. FAPO selects coordinated copy actions through a minimum vertex cut on $D^*(M)$. Each vertex is split into an input and an output vertex, with unit capacity on eligible vertex arcs and infinite capacity elsewhere. A supersource and supersink connect the critical sources and outputs. Any finite cut intersects every current critical path, and its capacity counts the selected copy actions. Minimizing this capacity therefore gives a smallest covering set within the fixed-DAG, same-cut family, when a feasible cover exists. A shared vertex can cover several reconvergent critical paths with one copy. Cutting the auxiliary graph identifies replication sites rather than removing LUTs from the circuit. Sink repartition then redistributes their loads.

In Fig. 1(b), $\{v, w\}$ covers all four longest paths; Fig. 1(c) illustrates sink repartition at a selected LUT. Added fanin loads can expose new bottlenecks, so coverage alone does not ensure delay reduction. CutBundle materializes the selected copies, remeasures timing, and rebuilds the critical DAG after accepted sets. Its bounded rounds preserve feasibility and nonincreasing delay, producing a candidate for joint area–timing evaluation.

TABLE II
SEARCH-STRATEGY COMPARISON AT $\beta = 5\%$. RATIOS ARE GEOMETRIC MEANS RELATIVE TO ACD-REMAP SEEDS AT MINIMUM ATP.

Search strategy	ATP ratio	T_1 ratio	A ratio
Greedy-only (matched)	0.8607	0.8549	1.0067
Minimum-cut-only	0.8757	0.8704	1.0061
Minimum-cut + region refinement	0.8753	0.8700	1.0061
50/50 greedy–region portfolio	0.8682	0.8627	1.0064
FAPO: greedy + one cut proposal	0.8553	0.8490	1.0074

C. Combining greedy refinement and cut proposals

FAPO combines greedy exploration of compound transformations with minimum-cut proposals that coordinate replication across the complete critical DAG. The two branches address complementary choices: greedy refinement couples sink assignment, cut switching, and recovery, while minimum cut identifies a smallest covering set in the restricted same-cut family.

Algorithm 1 maintains a search frontier $\mathcal{F}$ and a result archive $\mathcal{A}$. Greedy expansion advances the frontier, and the archive retains bounded nondominated area–delay tradeoffs. When $\alpha > 0$, $\beta > 0$, and $\mathcal{Q}$ is nonempty, one minimum-ATP mapping from a bounded Pareto sample supplies the base for CutBundle. A legal, area-feasible result enters the archive only if it strictly reduces delay relative to that base. Each sweep contributes at most one cut proposal, which may contain multiple copy actions; only the greedy branch advances the frontier.

This integration preserves compound exploration while adding structurally guided replication candidates. Final area recovery precedes minimum-ATP selection from the retained mappings and protected seed, so the selected result reflects both the realized timing benefit and the final LUT cost.

TABLE III
CRITICAL-PATH DELAY (CPD) AFTER PLACEMENT AND ROUTING. POSITIVE REDUCTIONS ARE BOLD. RED. DENOTES CPD REDUCTION.

Circuit	Parent (ns)	FAPo (ns)	Red. (%)	Circuit	Parent (ns)	FAPo (ns)	Red. (%)
adder	23.4439	22.5086	3.99	max	23.2156	21.0590	9.29
arbiter	12.1942	11.5675	5.14	mem_ctrl	16.9048	17.5559	-3.85
bar	3.8891	3.8546	0.89	multiplier	33.1260	31.1638	5.92
cavlc	2.8574	2.6597	6.92	priority	14.3388	13.8604	3.34
ctrl	1.6259	1.3909	14.46	router	4.6807	3.9507	15.60
dec	3.0451	2.5716	15.55	sin	25.3191	25.6171	-1.18
div	570.7600	536.2190	6.05	sqrt	588.4820	539.0380	8.40
i2c	3.6013	2.9589	17.84	square	25.2662	24.4143	3.37
int2float	2.6655	2.4523	8.00	voter	14.9015	13.5845	8.84
log2	53.0293	52.1067	1.74				
Geometric-mean CPD reduction (%)							7.04

IV. EXPERIMENTAL EVALUATION

A. Experiment Setup

The proposed FAPo framework is implemented in C++. All experiments were performed on a workstation equipped with an AMD Ryzen Threadripper PRO 7985WX processor (64 cores and 128 threads) and 128 GB of memory.

We used the EPFL benchmark suite [19]. ABC *if*, ACD-remap, and AGDMap-D provided the initial mappings and served as baselines [10], [11], [20]. Placement and routing were performed using VPR from VTR 9.0 with a 40-nm FPGA architecture [21].

B. Comparison with Initial Mappings

Unless otherwise stated, mapping experiments used $K = 6$, $\alpha = 1$, and $\beta = 5\%$. ABC *if* used the command sequence `strash; balance; if -K 6`. To test applicability across mapping flows, we applied the same FAPo policy to all three parents and selected minimum-ATP mappings within the per-circuit final-area cap. Table I compares each initial mapping with its refined counterpart; ratios are geometric means across the benchmarks. ATP decreased by 14.48%, 12.02%, and 12.04% for ACD-remap, ABC *if*, and AGDMap-D, respectively, with no circuit-level ATP regression. LUT-count increases remained within the 5% allowance for every circuit.

C. Search-Strategy Ablation

To assess minimum-cut proposals, we compared five strategies on eight ACD-remap development circuits (Table II). Greedy-only outperformed minimum-cut-only (ATP ratios 0.8607 versus 0.8757); FAPo achieved the lowest ratio, 0.8553. We then evaluated FAPo on the remaining 11 circuits. Against matched greedy-only search, FAPo/control ATP ratios were 0.9938 (development), 0.9941 (held-out), and 0.9940 (overall), giving a 0.60% additional gain. All strict improvements selected cut-derived mappings.

D. Effect of the Final-Area Allowance

To measure sensitivity to available area, we swept $\beta \in \{0, 1, 2, 5\}\%$ for all three parents, selecting minimum ATP independently at each allowance. Figure 2 compares ATP ratios to the corresponding initial mappings. Zero allowance

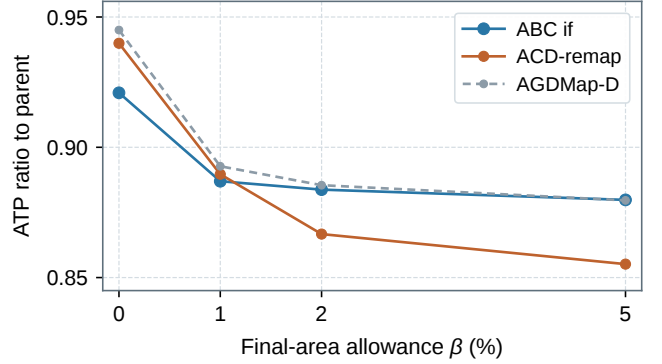


Fig. 2. Area-allowance sensitivity of FAPo. Each curve reports geometric-mean ATP ratios to its initial mapper on EPFL benchmarks; lower is better.

reduced ATP by 5.50%–7.91%; 1% yielded 10.73%–11.30% reductions. For ACD-remap, reductions reached 13.33% at 2% allowance and 14.48% at 5%, showing diminishing gains from additional area.

E. Post-Route Evaluation

To validate the physical timing benefits of FAPo, we evaluated ABC *if* mappings and their FAPo-refined counterparts through VPR placement and routing on the $K = 6$, $N = 10$ architecture. Both used identical devices, channel widths, and placement-and-routing settings. Table III reports a 7.04% CPD reduction on EPFL benchmarks. Reductions reached 17.84% for *i2c*, 15.60% for *router*, and 15.55% for *dec*. Packed CLB and wirelength ratios were 1.0080 and 1.0151, respectively.

V. CONCLUSION

FAPo jointly refines sink assignments and LUT cuts in a mapper-independent state, evaluating compound replication and recovery by final live area and timing. Its all-critical-path condition guides restricted minimum-copy proposals that complement greedy refinement. EPFL experiments across three initial mappers show up to 14.48% lower geometric-mean ATP with a 5% final-area allowance. VPR evaluation shows a 7.04% reduction in routed critical-path delay relative to ABC *if*.

REFERENCES

- [1] J. Cong and Y. Ding, “FlowMap: An optimal technology mapping algorithm for delay optimization in lookup-table based FPGA designs,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 13, no. 1, pp. 1–12, 1994.
- [2] —, “On area/depth trade-off in LUT-based FPGA technology mapping,” *IEEE Transactions on Very Large Scale Integration Systems*, vol. 2, no. 2, pp. 137–148, 1994.
- [3] A. Mishchenko, S. Cho, S. Chatterjee, and R. K. Brayton, “Combinational and sequential mapping with priority cuts,” in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*, 2007, pp. 354–361.
- [4] J. Cong, C. Wu, and Y. Ding, “Cut ranking and pruning: Enabling a general and efficient FPGA mapping solution,” in *Proceedings of the ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 1999, pp. 29–35.
- [5] A. Mishchenko, S. Chatterjee, and R. K. Brayton, “Improvements to technology mapping for LUT-based FPGAs,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 26, no. 2, pp. 240–253, 2007.
- [6] D. Chen and J. Cong, “DAOmap: A depth-optimal area optimization mapping algorithm for FPGA designs,” in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*, 2004, pp. 752–759.
- [7] J. Cong and Y. Ding, “On nominal delay minimization in LUT-based FPGA technology mapping,” *Integration, the VLSI Journal*, vol. 18, no. 1, pp. 73–94, 1994.
- [8] J. Y. Lin, A. Jagannathan, and J. Cong, “Placement-driven technology mapping for LUT-based FPGAs,” in *Proceedings of the ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2003, pp. 121–126.
- [9] S. Jang, B. Chan, K. Chung, and A. Mishchenko, “WireMap: FPGA technology mapping for improved routability and enhanced LUT merging,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 2, no. 2, pp. 14:1–14:24, 2009.
- [10] L. Fan and C. Wu, “FPGA technology mapping with adaptive gate decomposition,” in *Proceedings of the ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2023, pp. 135–140.
- [11] A. T. Calvino, G. D. Micheli, A. Mishchenko, and R. K. Brayton, “Enhancing delay-driven LUT mapping with boolean decomposition,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 3, pp. 1017–1030, 2025.
- [12] M. Hofmann, B. Gokmen, and Z. Zhang, “EqMap: FPGA LUT remapping using e-graphs,” in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*, 2025, pp. 1–9.
- [13] Z. Hu, H. Pan, Y. Xia, L. Wang, and Z. Chu, “Mixed structural choice operator: Enhancing technology mapping with heterogeneous representations,” in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*, 2025, pp. 1–7.
- [14] A. Costamagna, A. T. Calvino, A. Mishchenko, and G. D. Micheli, “Area-oriented resubstitution for networks of look-up tables,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 7, pp. 2571–2584, 2025.
- [15] C. Yu, “Mapping fusion: Improving FPGA technology mapping with ASIC mapper,” in *Proceedings of the ACM/IEEE Symposium on Machine Learning for CAD*, 2025, pp. 1–7.
- [16] C.-H. Chen and C.-Y. Tsui, “Timing optimization of logic network using gate duplication,” in *Proceedings of the Asia and South Pacific Design Automation Conference*, 1999, pp. 233–236.
- [17] G. Beraudo and J. Lillis, “Timing optimization of FPGA placements by logic replication,” in *Proceedings of the ACM/IEEE Design Automation Conference*, 2003, pp. 196–201.
- [18] A. Mishchenko, S. Chatterjee, R. K. Brayton, and N. Eén, “Improvements to combinational equivalence checking,” in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*, 2006, pp. 836–843.
- [19] L. Amarù, P.-E. Gaillardon, and G. D. Micheli, “The EPFL combinational benchmark suite,” in *Proceedings of the International Workshop on Logic and Synthesis*, 2015. [Online]. Available: <https://github.com/lils/benchmarks>
- [20] R. K. Brayton and A. Mishchenko, “ABC: An academic industrial-strength verification tool,” in *Computer Aided Verification*, ser. Lecture Notes in Computer Science, vol. 6174, 2010, pp. 24–40.
- [21] M. A. Elgammal, A. Mohaghegh, S. G. Shahrouz, F. Mahmoudi, F. Koşar, K. Talaei, J. Fife, D. Khadivi, K. Murray, A. Boutros, K. B. Kent, J. Goeters, and V. Betz, “VTR 9: Open-source CAD for fabric and beyond FPGA architecture exploration,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 18, no. 3, pp. 1–53, 2025.